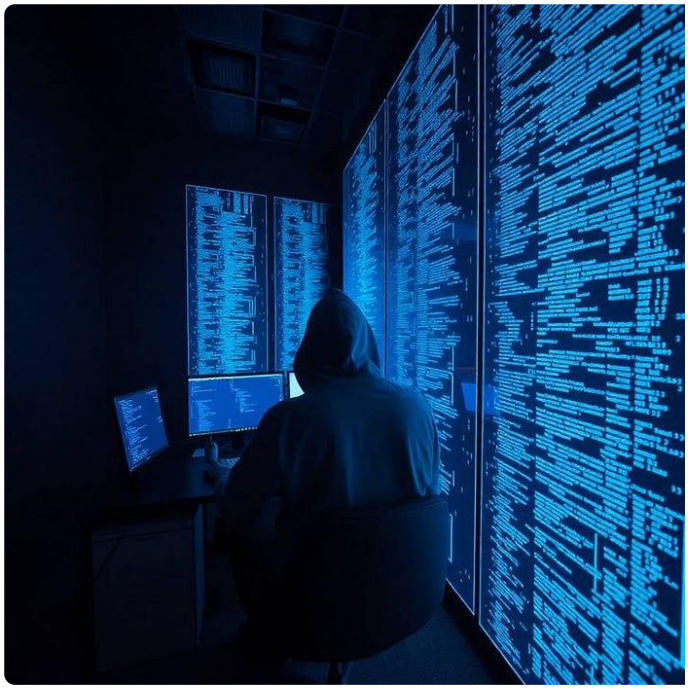


Hackers Mapping your Complete Network in Just Some Minutes — The Process Revealed

Phase 1: Port Scanning with Nmap

SAYEM-EH

Published June 8, 2026

Free: Yes

Read on Freedium · original

Contents

Phase 1: Port Scanning with Nmap 0

Phase 2: Banner Grabbing 0

When Banners Are Missing or Fake 0

Phase 3: Building a Service Inventory 0

Phase 4: Matching Versions to Known Vulnerabilities 0

The Databases You Need to Know 0

The Workflow 0

Understanding Severity Ratings (And Their Limitations) 0

The Gap Between "Vulnerable" and "Exploitable" 0

Automated Scanners: Useful, Not Sufficient 0

Putting It All Together: The Enumeration Checklist 0

Why This Matters Even If You're Not a Pentester 0

Phase 1: Port Scanning with Nmap

An Nmap is usually how any security evaluation begins. Packets are sent by the program to certain ports of a target, and it tells you which ports respond. The biggest mistake that beginners make is thinking that simply finding out what port responds means something. Port 80 is traditionally reserved for HTTP communication. Port 22 is traditionally reserved for SSH connections. Port 443 is traditionally HTTPS port. But there is no rule that says that you cannot run SSH service on port 80, or hide your webserver on port 8443, or run any software on any port at all. And assuming that it is done the traditional way will only take your time. Which is why we not only scan, but probe.

Here's the command I run at the start of every engagement:

```
nmap -sV -sC -p- TARGET_IP -oN scan_results.Txt
```

Let me break down every flag:

`-sV` (Version Detection)

This is the most significant flag. Instead of just saying "port 22 is open," Nmap goes one step further by testing the port and trying to detect not only the name of the software but its version too. The distinction between "port 22 is open" and "port 22 is running OpenSSH 7.6p1 on Ubuntu" is huge.

`-sC` (Default Scripts)

This will run Nmap's own script engine on each identified service. By default, the scripts conduct some useful probing operations automatically, such as anonymous FTP login, HTTP page title retrieval, SMB signing status detection, and many more. In essence, think of it as follow-up questions asked to each identified service.

`-p-` (All Ports)

By default, Nmap will only scan the top 1,000 most common ports. This leaves another 64,535 ports unscanned. Services that run on the other ports would never even be noticed. The -p- option allows us to scan all of the 65,535 TCP ports. It is slower but well worth it.

`-oN scan_results` (Save Output)

Results saved into a file. You will be referencing this output throughout your entire penetration test. Scrolling up in the terminal is never an option.

This output gives you the protocol, the service, and where Nmap knows how to figure out both, along with the version number of the software that is running on that open port. The version numbers are the most valuable piece of output from the entire enumeration process.

Phase 2: Banner Grabbing

The process of banner grabbing is relatively simple compared to Nmap scanning. You just connect to the port and see what it tells you at first. Most services immediately introduce themselves after connection by providing their name and version details. Here's how you can do this with netcat:

```
nc TARGET_IP 22
```

And you'll see something like:

```
SSH-2.0-OpenSSH_7.6p1 Ubuntu-4ubuntu0.3
```

That single line just told you three things:

1. The SSH protocol version (2.0)
2. The exact OpenSSH version (7.6p1)
3. The underlying operating system (Ubuntu)

That's enough to start searching for known vulnerabilities against that specific version. One connection. One line of output. Significant intelligence.

Try it against an SMTP service:

```
nc TARGET_IP 25
220 mail.target.local ESMTP Postfix (Ubuntu)
```

Now you know the mail server software (Postfix), the hostname (mail.target.local), and the OS (Ubuntu). You can perform banner grabbing manually with netcat or telnet, or let Nmap's version detection probes handle it automatically.

When Banners Are Missing or Fake

All services do not cooperate in this way. Some are designed to strip the version information and reply only with a general message. The workaround provided by Nmap is service fingerprinting. Nmap will send specially crafted requests and compare the reply with a database of known signatures for that service. It may not work as well as a direct read of the banner, but it does narrow things down. It is

also possible to manipulate the banners to provide misleading information. Security-conscious administrators might alter version information. While rare outside of honeypots, this trick doesn't always fool other methods of detecting the service type anyway. Bottom line: banners offer the quickest access to the version information but are not the only access method available.

Phase 3: Building a Service Inventory

As you enumerate each port, build an inventory. This is a working document, not a final report. For each service, record the port number, protocol, software name and version, and any extra details revealed by scripts or manual inspection.

Here's what a typical service inventory looks like after the first pass:

Press enter or click to view image in full size

Every column matters. The port tells you where. The service tells you what. The version tells you which. And the notes capture context that might be relevant later.

"SMB signing not required" might seem like a minor detail now. During exploitation, it's the difference between a relay attack working or not. "Remote connections enabled" on MySQL means you might be able to authenticate directly if you find credentials elsewhere. These notes accumulate into a map of the target's attack surface.

This inventory is a living document. You'll annotate it, update it, cross out dead ends, and highlight promising targets as the engagement progresses.

Phase 4: Matching Versions to Known Vulnerabilities

Once you have identified all the services, the hard part comes into play. You have to figure out which of those services have any known vulnerabilities. You do this by finding any matching vulnerabilities based on their versions and seeing if that information is relevant to your target.

The Databases You Need to Know

CVE (Common Vulnerabilities and Exposures)

Identification System One. The entries here describe a particular problem in a specific software package. This entry is identified using the standard CVE-YYYY-NNNNN form. Whenever a security professional refers to a problem, they do so by referring to the CVE number. It's the lingua franca.

NVD (National Vulnerability Database)

Maintained by NIST, the NVD adds severity scores, technical detail, and references to each CVE. It's where you go for the full picture: what the vulnerability is, how severe it is, and what conditions are required to exploit it.

Exploit-DB

This takes it one step further by linking CVE entries to publicly available exploit code. This gives you an immediate sense of whether a flaw is exploitable in practice, not just in theory. If exploit code is working on Exploit-DB, then the vulnerability is already proven to work. Searchsploit allows you to search Exploit-DB from the command line against a local copy of the database:

```
searchsploit apache 2.4.29
```

This returns matching entries along with the file path to the exploit code on your system. No browser required. Instant results.

GitHub

Often the fastest source, especially for recent disclosures. When a high-profile CVE drops, proof-of-concept exploit code frequently appears on GitHub within hours. Security researchers and offensive tooling developers publish working PoCs, sometimes before the vulnerability even makes it into Exploit-DB. Searching GitHub for a CVE identifier directly, like `CVE-2021-41773`, will often turn up multiple repositories containing exploit scripts, detailed write-ups, and scanning tools.

Check GitHub alongside the traditional databases, especially for recently disclosed vulnerabilities where Exploit-DB may not yet have an entry.

The Workflow

For each service in your inventory:

1. Take the software name and version from your inventory
2. Search the NVD or Exploit-DB for that software and version
3. Check GitHub for the CVE identifier to find proof-of-concept code or tooling
4. Review the results, noting any CVEs affecting the exact version or a range that includes it
5. Assess each result for relevance and exploitability
6. Repeat for every service in the inventory

This is methodical work. It's not glamorous. But it's where you find the entries that lead to access.

Understanding Severity Ratings (And Their Limitations)

Most CVE entries carry a CVSS score, rating severity from 0.0 to 10.0. The score accounts for the attack vector (network vs. local access), whether authentication is required, and the impact on confidentiality, integrity, and availability.

A quick reference:

Press enter or click to view image in full size

A critical-severity vulnerability allowing unauthenticated remote code execution is a fundamentally different finding from a low-severity information disclosure issue that requires local access. That much is obvious.

But here's what's less obvious: **CVSS scores should not be treated as the only measure of usefulness during an engagement.**

If you can't meet the prerequisites of the critical vulnerability, then it's not necessarily more important than a medium-severity vulnerability that allows you to get authenticated access to a system. A "low" severity information disclosure leaking credentials is more useful than a "critical" buffer overflow that requires a specific kernel version the target is not running. The scores give you a starting point for prioritization. You get the real priority in context.

The Gap Between "Vulnerable" and "Exploitable"

This is the concept that separates junior testers from experienced ones.

A CVE might describe a buffer overflow in a particular version of a web server. Your scan shows the target is running that exact version. Looks like a hit. But consider:

The target may have applied a patch without updating the version string. Many organizations patch vulnerabilities through backporting, fixing the flaw without upgrading the entire software version. The version number stays the same. The vulnerability is gone. Your scan can't tell the difference.

The flaw may require a specific module or configuration. A vulnerability in Apache's `mod_cgi` is irrelevant if the target doesn't have `mod_cgi` enabled. A flaw that requires WebDAV is useless if WebDAV isn't configured. The vulnerability exists in the software, but not in this deployment of it.

Network-level controls may block the attack. A firewall, WAF, or IPS might prevent the traffic needed to trigger the vulnerability from ever reaching the service.

The exploit may require valid credentials. A vulnerability that allows privilege escalation from an authenticated user is powerful, but only if you've already obtained credentials. If you haven't, it sits in your notes as "potential, pending access."

The exploit may depend on a specific environment. A memory corruption exploit that works reliably on x86 might fail on ARM. An exploit that requires a specific library version might not work if the target has a different build.

The filtering looks like this:

```
All discovered services
   ↓ filter
Services with known CVEs
   ↓ filter
CVEs matching the target's configuration
   ↓ filter
Actually exploitable vulnerabilities
```

Each filter narrows the list significantly. You might start with 50 services, find CVEs for 15 of them, determine that 8 match the target's configuration, and conclude that 3 are actually exploitable. Those 3 are your prioritized targets.

This is why vulnerability identification is a separate phase from exploitation. The goal here is to build a prioritized list of potential weaknesses, not to confirm each one. Confirmation happens when you try the exploit.

Automated Scanners: Useful, Not Sufficient

Tools like Nessus, OpenVAS, and Nikto automate the process of matching services against vulnerability databases. They run their own enumeration, compare results against known issues, and produce a report of potential vulnerabilities.

These scanners are genuinely useful for covering a broad attack surface quickly. In a time-constrained engagement, they catch things you might miss during manual review. They're particularly good at finding well-known, well-documented vulnerabilities across large numbers of hosts.

But they have real limitations:

False positives. Scanners flag everything that might be a vulnerability. A significant percentage of their findings will be incorrect, triggered by version string matching without deeper verification.

Missed context. Scanners can't understand business logic flaws, chained vulnerabilities that only work in combination, or weaknesses that require understanding how the application actually behaves. They test individual services in isolation.

No adaptation. A scanner runs the same checks regardless of what it finds. A human tester adjusts their approach based on each new piece of information. A scanner that finds MySQL credentials in an exposed config file won't think to use those credentials against the admin panel on a different port.

Scanner output is a starting point for manual investigation. Not a replacement for it. Use them for coverage. Trust your own analysis for depth.

Putting It All Together: The Enumeration Checklist

Here's the complete workflow from first scan to prioritized target list:

1. Full port scan with version detection

```
nmap -sV -sC -p- TARGET_IP -oN scan_results
```

2. Manual banner grabbing on interesting ports

```
nc TARGET_IP 22
nc TARGET_IP 25
nc TARGET_IP 80
```

3. Build the service inventory

Record every service with its port, software, version, and notes.

4. Search vulnerability databases for each service

```
searchsploit [software] [version]
```

Then check NVD and GitHub for each CVE.

5. Filter for relevance

Does the vulnerability match the target's configuration? Are the prerequisites satisfiable? Is exploit code available?

6. Prioritize

Rank the remaining vulnerabilities by exploitability and impact, not just CVSS score.

7. Document everything

Your service inventory, your vulnerability matches, your filtering rationale, and your prioritized list. This is the document you'll work from during exploitation and the document you'll reference when writing your report.

Why This Matters Even If You're Not a Pentester

For developers and system administrators, the understanding of enumeration changes how you think about your own infrastructure. Every service you leave running is a surface area. Every unpatched version is a potential hit in a vulnerability database. Each banner you broadcast is information you are giving out to anyone who connects. Run this scan on your own systems. Develop the inventory. Find your own versions in Exploit-DB. You might be surprised at what you discover. The attackers won't.